Scalability Risks

Highest-risk bottlenecks

1. Process-local realtime state

- online users, offline grace timers, livestream dedupe windows, participant sets, and host disconnect timers are all in memory
- multi-instance deployment will cause inconsistent presence, viewer counts, and livestream state

2. Monolithic runtime

- API traffic, Socket.IO, static web, store pages, blog pages, and scheduled jobs all share one Node process
- unrelated load can interfere across domains

3. Request-path fan-out

- notification delivery and community-post fan-out happen close to request paths
- large communities or bursty activity can increase user-facing latency

4. Full-table scheduled work

- daily verification and reward jobs iterate the full user set
- monthly resets snapshot all users

5. Dynamic feed assembly

- country scoping, privacy filtering, populates, liked-state decoration, and ad injection are all request-time operations

Secondary risks

- duplicated permission logic can cause policy drift under scale
- denormalized counters can drift and force repair work
- moderation read endpoints performing writes increase operational surprise

Recommended mitigations

1. add Redis for Socket.IO adapter, presence, and livestream room state
2. split cron workers from the web server
3. queue high-fan-out notifications
4. introduce feed caching or precomputation
5. standardize one RBAC model and one event schema